Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «интеллектуальный анализ данных»
Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:
Студент 4 курса
Группы ИИ-25
Пальчастый Я.М
Проверил:
Андренко К.В.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью RBM

Вариант 8:

8	https://archive.ics.uci.edu/dataset/162/forest+fires	регрессия	area

Ход работы

```
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import tensorflow as tf
import keras
from keras import layers, models, callbacks, optimizers
from keras.initializers import Constant
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, MinMaxScaler
from sklearn.compose import ColumnTransformer
from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                             mean_absolute_percentage_error,
                             accuracy_score, f1_score,
                             confusion_matrix, classification_report)
from scipy.io import arff
import ucimlrepo


SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)


def choose_dataset():
    print("Выбор датасета")
    print("1 - Forest Fires (регрессия)")
    print("2 - Rice (классификация)")
    while True:
        try:
            choice = input("Введите номер датасета (1 или 2): ").strip()
        except (EOFError, KeyboardInterrupt):
            return "classification"
        if choice == "1":
            print("Выбран Forest Fires")
            return "regression"
        if choice == "2":
            print("Выбран Rice")
            return "classification"
        print("Некорректный ввод.")


def load_forest_fires(path="forestfires.csv"):
```

```
    if not os.path.exists(path):
        url = ("https://archive.ics.uci.edu/ml/machine-learning-databases/"
               "forest-fires/forestfires.csv")
        df = pd.read_csv(url)
        df.to_csv(path, index=False)
    else:
        df = pd.read_csv(path)
    return df


def load_rice(path="Rice_Cammeo_Osmancik.arff"):
    if os.path.exists(path):
        data, meta = arff.loadarff(path)
        df = pd.DataFrame(data)
    else:
        dataset = ucimlrepo.fetch_ucirepo(id=545)
        X = dataset.data.features.copy()
        y = dataset.data.targets.copy()
        if isinstance(y, pd.DataFrame):
            y = y.iloc[:, 0]
        df = X.copy()
        df["Class"] = y.values
        df.to_csv("Rice_Cammeo_Osmancik.csv", index=False)
    for col in df.columns:
        if df[col].dtype == object:
            df[col] = df[col].apply(
                lambda v: v.decode("utf-8") if isinstance(v, bytes) else v)
    return df


def preprocess_regression(df):
    y = np.log1p(df["area"].values.astype(np.float64))
    X = df.drop(columns=["area"])
    categorical_features = ["month", "day"]
    numeric_features = ["X", "Y", "FFMC", "DMC", "DC",
                        "ISI", "temp", "RH", "wind", "rain"]
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), numeric_features),
            ("cat", OneHotEncoder(handle_unknown="ignore"),
             categorical_features),
        ])
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=SEED)
    X_train_prep = preprocessor.fit_transform(X_train).astype(np.float32)
    X_test_prep = preprocessor.transform(X_test).astype(np.float32)
    return (X_train_prep, X_test_prep,
            y_train.astype(np.float32), y_test.astype(np.float32))


def preprocess_classification(df, target_col="Class"):
    X = df.drop(columns=[target_col])
    y_raw = df[target_col].values
    classes = np.unique(y_raw)
    class_to_idx = {c: i for i, c in enumerate(classes)}
    num_classes = len(classes)
    y_idx = np.array([class_to_idx[c] for c in y_raw], dtype=np.int32)
    y_onehot = tf.keras.utils.to_categorical(y_idx, num_classes=num_classes)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y_onehot, test_size=0.2, random_state=SEED, stratify=y_idx)
    scaler = StandardScaler()
    X_train_prep = scaler.fit_transform(X_train).astype(np.float32)
    X_test_prep = scaler.transform(X_test).astype(np.float32)
    y_train_labels = np.argmax(y_train, axis=1)
    y_test_labels = np.argmax(y_test, axis=1)
```

```
    return (X_train_prep, X_test_prep, y_train, y_test,
            classes, num_classes, y_train_labels, y_test_labels)


def to_unit_interval(X_train, X_test):
    s = MinMaxScaler()
    return (s.fit_transform(X_train).astype(np.float32),
            s.transform(X_test).astype(np.float32))


def evaluate_regression(model, X_test, y_test, name):
    y_pred_log = model.predict(X_test, verbose=0).flatten()
    y_pred = np.expm1(y_pred_log)
    y_true = np.expm1(y_test)
    mse = mean_squared_error(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    mask = y_true != 0
    mape = mean_absolute_percentage_error(y_true[mask], y_pred[mask]) * 100.0
    print(name)
    print("MSE:", mse)
    print("MAE:", mae)
    print("MAPE:", mape)
    return {"mse": mse, "mae": mae, "mape": mape,
            "y_pred": y_pred, "y_true": y_true}


def evaluate_classification(model, X_test, y_test_onehot, y_test_labels,
                            class_names, name):
    y_pred_proba = model.predict(X_test, verbose=0)
    y_pred_labels = np.argmax(y_pred_proba, axis=1)
    acc = accuracy_score(y_test_labels, y_pred_labels)
    f1_macro = f1_score(y_test_labels, y_pred_labels,
                        average="macro", zero_division=0)
    f1_weighted = f1_score(y_test_labels, y_pred_labels,
                           average="weighted", zero_division=0)
    print(name)
    print("Accuracy:", acc)
    print("F1 macro:", f1_macro)
    print("F1 weighted:", f1_weighted)
    print(classification_report(y_test_labels, y_pred_labels,
                                target_names=[str(c) for c in class_names],
                                zero_division=0))
    print(confusion_matrix(y_test_labels, y_pred_labels))
    return {"accuracy": acc, "f1_macro": f1_macro, "f1_weighted": f1_weighted,
            "y_pred": y_pred_labels, "y_true": y_test_labels}


def build_baseline_regression(input_dim):
    model = models.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu", name="h1"),
        layers.Dense(32, activation="relu", name="h2"),
        layers.Dense(16, activation="relu", name="h3"),
        layers.Dense(1, name="out"),
    ])
    model.compile(optimizer=optimizers.Adam(learning_rate=1e-3),
                  loss="mse", metrics=["mae"])
    return model


def build_baseline_classification(input_dim, num_classes):
    model = models.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu", name="h1"),
        layers.Dense(32, activation="relu", name="h2"),
```

```
        layers.Dense(16, activation="relu", name="h3"),
        layers.Dense(num_classes, activation="softmax", name="out"),
    ])
    model.compile(optimizer=optimizers.Adam(learning_rate=1e-3),
                  loss="categorical_crossentropy", metrics=["accuracy"])
    return model


def train_baseline(X_train, y_train, input_dim, task,
                   num_classes=None, epochs=200, batch_size=32):
    if task == "regression":
        model = build_baseline_regression(input_dim)
    else:
        model = build_baseline_classification(input_dim, num_classes)
    early = callbacks.EarlyStopping(monitor="val_loss", patience=25,
                                    restore_best_weights=True)
    history = model.fit(X_train, y_train, validation_split=0.1,
                        epochs=epochs, batch_size=batch_size,
                        verbose=0, callbacks=[early])
    return model, history


def train_autoencoder(X, hidden_dim, epochs=100, batch_size=32,
                      learning_rate=1e-3):
    input_dim = X.shape[1]
    input_layer = layers.Input(shape=(input_dim,))
    encoded = layers.Dense(hidden_dim, activation="relu",
                           name="encoder")(input_layer)
    decoded = layers.Dense(input_dim, activation="linear",
                           name="decoder")(encoded)
    autoencoder = models.Model(input_layer, decoded)
    autoencoder.compile(optimizer=optimizers.Adam(learning_rate=learning_rate),
                        loss="mse")
    early = callbacks.EarlyStopping(monitor="loss", patience=15,
                                    restore_best_weights=True)
    autoencoder.fit(X, X, epochs=epochs, batch_size=batch_size,
                    verbose=0, callbacks=[early])
    encoder = models.Model(input_layer, encoded)
    return encoder


def pretrain_autoencoder_layerwise(X_train, hidden_dims=(64, 32, 16),
                                   epochs_per_layer=100, batch_size=32):
    encoders = []
    current = X_train
    for i, dim in enumerate(hidden_dims):
        print("Предобучение слоя", i + 1, "dim =", dim)
        encoder = train_autoencoder(current, hidden_dim=dim,
                                    epochs=epochs_per_layer,
                                    batch_size=batch_size)
        encoders.append(encoder)
        current = encoder.predict(current, verbose=0)
        print("Выход автоэнкодера:", current.shape)
    return encoders


def build_pretrained_ae(encoders, input_dim, task, num_classes=None):
    model = models.Sequential()
    model.add(layers.Input(shape=(input_dim,)))
    for i, enc in enumerate(encoders):
        w, b = enc.layers[1].get_weights()
        model.add(layers.Dense(w.shape[1], activation="relu",
                               kernel_initializer=Constant(w),
                               bias_initializer=Constant(b),
                               name=f"h{i+1}"))
```

```
    if task == "regression":
        model.add(layers.Dense(1, name="out"))
        model.compile(optimizer=optimizers.Adam(1e-3),
                      loss="mse", metrics=["mae"])
    else:
        model.add(layers.Dense(num_classes, activation="softmax", name="out"))
        model.compile(optimizer=optimizers.Adam(1e-3),
                      loss="categorical_crossentropy", metrics=["accuracy"])
    return model


def train_with_ae_pretraining(X_train, y_train, input_dim, task,
                              num_classes=None, hidden_dims=(64, 32, 16),
                              epochs_per_layer=100, fine_tune_epochs=200,
                              batch_size=32):
    encoders = pretrain_autoencoder_layerwise(
        X_train, hidden_dims=hidden_dims,
        epochs_per_layer=epochs_per_layer, batch_size=batch_size)
    model = build_pretrained_ae(encoders, input_dim, task, num_classes)
    early = callbacks.EarlyStopping(monitor="val_loss", patience=25,
                                    restore_best_weights=True)
    history = model.fit(X_train, y_train, validation_split=0.1,
                        epochs=fine_tune_epochs, batch_size=batch_size,
                        verbose=0, callbacks=[early])
    return model, history


def _sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -50, 50)))


class RBM:
    def __init__(self, n_visible, n_hidden, seed=42):
        rng = np.random.RandomState(seed)
        self.W = (rng.normal(0.0, 0.01,
                             (n_visible, n_hidden))).astype(np.float32)
        self.bv = np.zeros(n_visible, dtype=np.float32)
        self.bh = np.zeros(n_hidden, dtype=np.float32)

    def _p_h_given_v(self, v):
        return _sigmoid(v @ self.W + self.bh)

    def _p_v_given_h(self, h):
        return _sigmoid(h @ self.W.T + self.bv)

    def _sample_h(self, v):
        p = self._p_h_given_v(v)
        return (p > np.random.rand(*p.shape)).astype(np.float32), p

    def _sample_v(self, h):
        p = self._p_v_given_h(h)
        return (p > np.random.rand(*p.shape)).astype(np.float32), p

    def _gibbs_chain(self, v0, k):
        h0, ph0 = self._sample_h(v0)
        vk, hk = v0, h0
        for _ in range(k):
            vk, _ = self._sample_v(hk)
            hk, _ = self._sample_h(vk)
        phk = self._p_h_given_v(vk)
        return h0, ph0, vk, phk

    def cd_step(self, v0, lr, k=1):
        h0, ph0, vk, phk = self._gibbs_chain(v0, k)
        n = v0.shape[0]
```

```
        self.W += lr * ((v0.T @ ph0) / n - (vk.T @ phk) / n)
        self.bv += lr * (v0.mean(axis=0) - vk.mean(axis=0))
        self.bh += lr * (ph0.mean(axis=0) - phk.mean(axis=0))

    def transform(self, X):
        return self._p_h_given_v(X.astype(np.float32))


def pretrain_rbm_layerwise(X_train, hidden_dims=(64, 32, 16),
                           epochs=100, lr=0.01, batch_size=32, k=1):
    rbms, current = [], X_train.astype(np.float32)
    for i, dim in enumerate(hidden_dims):
        print(f"RBM-предобучение слоя {i+1} dim = {dim}")
        rbm = RBM(current.shape[1], dim, seed=SEED + i)
        n = current.shape[0]
        for _ in range(epochs):
            idx = np.random.permutation(n)
            for start in range(0, n, batch_size):
                batch = current[idx[start:start + batch_size]]
                rbm.cd_step(batch, lr=lr, k=k)
        rbms.append(rbm)
        current = rbm.transform(current)
        print(f"  выход RBM: {current.shape}")
    return rbms


def build_pretrained_rbm(rbms, input_dim, task, num_classes=None,
                         hidden_activation="relu"):
    """
    Собирает итоговую сеть на основе предобученных RBM-слоёв.
    hidden_activation: "relu" (по умолчанию) или "sigmoid".
    ReLU убирает насыщение активации и позволяет fine-tuning реально
    изменять веса после RBM-инициализации.
    """
    model = models.Sequential()
    model.add(layers.Input(shape=(input_dim,)))
    for i, rbm in enumerate(rbms):
        model.add(layers.Dense(
            rbm.W.shape[1],
            activation=hidden_activation,
            kernel_initializer=Constant(rbm.W),
            bias_initializer=Constant(rbm.bh),
            name=f"h{i+1}",
        ))
    if task == "regression":
        model.add(layers.Dense(1, name="out"))
        model.compile(optimizer=optimizers.Adam(1e-3),
                      loss="mse", metrics=["mae"])
    else:
        model.add(layers.Dense(num_classes, activation="softmax", name="out"))
        model.compile(optimizer=optimizers.Adam(1e-3),
                      loss="categorical_crossentropy", metrics=["accuracy"])
    return model


def train_with_rbm_pretraining(X_train, y_train, input_dim, task,
                               num_classes=None, hidden_dims=(64, 32, 16),
                               epochs_per_layer=100, lr_rbm=0.01,
                               batch_size=32, k_cd=1, fine_tune_epochs=200,
                               hidden_activation="relu"):
    rbms = pretrain_rbm_layerwise(X_train, hidden_dims=hidden_dims,
                                  epochs=epochs_per_layer, lr=lr_rbm,
                                  batch_size=batch_size, k=k_cd)
    model = build_pretrained_rbm(rbms, input_dim, task, num_classes,
                                 hidden_activation=hidden_activation)
```

```
    early = callbacks.EarlyStopping(monitor="val_loss", patience=25,
                                    restore_best_weights=True)
    history = model.fit(X_train, y_train, validation_split=0.1,
                        epochs=fine_tune_epochs, batch_size=batch_size,
                        verbose=0, callbacks=[early])
    return model, history


def plot_three_histories(h_no, h_ae, h_rbm, task, save_path):
    fig, axes = plt.subplots(1, 3, figsize=(16, 4))
    ylabel = "MSE" if task == "regression" else "Categorical CE"
    for ax, h, title in zip(axes, [h_no, h_ae, h_rbm],
                            ["Без предобучения", "Автоэнкодер", "RBM"]):
        ax.plot(h.history["loss"], label="train")
        ax.plot(h.history["val_loss"], label="val")
        ax.set_title(title)
        ax.set_xlabel("Эпоха")
        ax.set_ylabel(ylabel)
        ax.legend()
        ax.grid(True)
    plt.tight_layout()
    plt.savefig(save_path, dpi=150)
    plt.close()
    print("Сохранено:", save_path)


def plot_confusion(res_no, res_ae, res_rbm, class_names, save_path):
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    for ax, res, title in zip(axes, [res_no, res_ae, res_rbm],
                              ["Без предобучения", "Автоэнкодер", "RBM"]):
        cm = confusion_matrix(res["y_true"], res["y_pred"])
        im = ax.imshow(cm, interpolation="nearest", cmap=plt.cm.Blues)
        ax.set_title(title)
        ax.set_xlabel("Предсказанный класс")
        ax.set_ylabel("Истинный класс")
        ax.set_xticks(range(len(class_names)))
        ax.set_yticks(range(len(class_names)))
        ax.set_xticklabels(class_names, rotation=45)
        ax.set_yticklabels(class_names)
        plt.colorbar(im, ax=ax)
        thresh = cm.max() / 2.0
        for i in range(cm.shape[0]):
            for j in range(cm.shape[1]):
                ax.text(j, i, str(cm[i, j]), ha="center", va="center",
                        color="white" if cm[i, j] > thresh else "black")
    plt.tight_layout()
    plt.savefig(save_path, dpi=150)
    plt.close()
    print("Сохранено:", save_path)


def run_regression():
    df = load_forest_fires()
    X_train, X_test, y_train, y_test = preprocess_regression(df)
    X_train, X_test = to_unit_interval(X_train, X_test)
    input_dim = X_train.shape[1]
    print("Датасет: Forest Fires")
    print("Обучающая:", X_train.shape)
    print("Тестовая:", X_test.shape)
    print("Размерность:", input_dim)

    print("Шаг 1. Без предобучения")
    model_no, hist_no = train_baseline(X_train, y_train, input_dim,
                                       task="regression",
                                       epochs=200, batch_size=32)
```

```
    res_no = evaluate_regression(model_no, X_test, y_test,
                                 "Без предобучения")

    print("Шаг 2. Автоэнкодерное предобучение")
    model_ae, hist_ae = train_with_ae_pretraining(
        X_train, y_train, input_dim, task="regression",
        hidden_dims=(64, 32, 16), epochs_per_layer=100,
        fine_tune_epochs=200, batch_size=32)
    res_ae = evaluate_regression(model_ae, X_test, y_test, "Автоэнкодер")

    print("Шаг 3. RBM-предобучение")
    model_rbm, hist_rbm = train_with_rbm_pretraining(
        X_train, y_train, input_dim, task="regression",
        hidden_dims=(64, 32, 16), epochs_per_layer=100,
        lr_rbm=0.01, batch_size=32, k_cd=1, fine_tune_epochs=200,
        hidden_activation="relu")
    res_rbm = evaluate_regression(model_rbm, X_test, y_test, "RBM")

    plot_three_histories(hist_no, hist_ae, hist_rbm, "regression",
                         "training_curves_forest.png")

    df_summary = pd.DataFrame({
        "Метрика": ["MSE", "MAE", "MAPE, %"],
        "Без предобучения": [res_no["mse"], res_no["mae"], res_no["mape"]],
        "Автоэнкодер":      [res_ae["mse"], res_ae["mae"], res_ae["mape"]],
        "RBM":              [res_rbm["mse"], res_rbm["mae"], res_rbm["mape"]],
    })
    df_summary.to_csv("summary_forest.csv", index=False)
    print(df_summary.to_string(index=False))


def run_classification():
    df = load_rice()
    (X_train, X_test, y_train, y_test, classes, num_classes,
     y_train_labels, y_test_labels) = preprocess_classification(df)
    X_train, X_test = to_unit_interval(X_train, X_test)
    input_dim = X_train.shape[1]
    class_names = [str(c) for c in classes]
    print("Датасет: Rice")
    print("Обучающая:", X_train.shape)
    print("Тестовая:", X_test.shape)
    print("Размерность:", input_dim)
    print("Классов:", num_classes, class_names)

    print("Шаг 1. Без предобучения")
    model_no, hist_no = train_baseline(X_train, y_train, input_dim,
                                       task="classification",
                                       num_classes=num_classes,
                                       epochs=200, batch_size=64)
    res_no = evaluate_classification(model_no, X_test, y_test, y_test_labels,
                                     class_names, "Без предобучения")

    print("Шаг 2. Автоэнкодерное предобучение")
    model_ae, hist_ae = train_with_ae_pretraining(
        X_train, y_train, input_dim, task="classification",
        num_classes=num_classes, hidden_dims=(64, 32, 16),
        epochs_per_layer=100, fine_tune_epochs=200, batch_size=64)
    res_ae = evaluate_classification(model_ae, X_test, y_test, y_test_labels,
                                     class_names, "Автоэнкодер")

    print("Шаг 3. RBM-предобучение")
    model_rbm, hist_rbm = train_with_rbm_pretraining(
        X_train, y_train, input_dim, task="classification",
        num_classes=num_classes, hidden_dims=(64, 32, 16),
        epochs_per_layer=100, lr_rbm=0.01, batch_size=64,
```

```
        k_cd=1, fine_tune_epochs=200, hidden_activation="relu")
    res_rbm = evaluate_classification(model_rbm, X_test, y_test, y_test_labels,
                                      class_names, "RBM")

    plot_three_histories(hist_no, hist_ae, hist_rbm, "classification",
                         "training_curves_rice.png")
    plot_confusion(res_no, res_ae, res_rbm, class_names,
                   "confusion_matrices.png")

    df_summary = pd.DataFrame({
        "Метрика": ["Accuracy", "F1 (macro)", "F1 (weighted)"],
        "Без предобучения": [res_no["accuracy"], res_no["f1_macro"],
                             res_no["f1_weighted"]],
        "Автоэнкодер":      [res_ae["accuracy"], res_ae["f1_macro"],
                             res_ae["f1_weighted"]],
        "RBM":              [res_rbm["accuracy"], res_rbm["f1_macro"],
                             res_rbm["f1_weighted"]],
    })
    df_summary.to_csv("summary_rice.csv", index=False)
    print(df_summary.to_string(index=False))


def main():
    task = choose_dataset()
    if task == "regression":
        run_regression()
    else:
        run_classification()


if __name__ == "__main__":
    main()
```

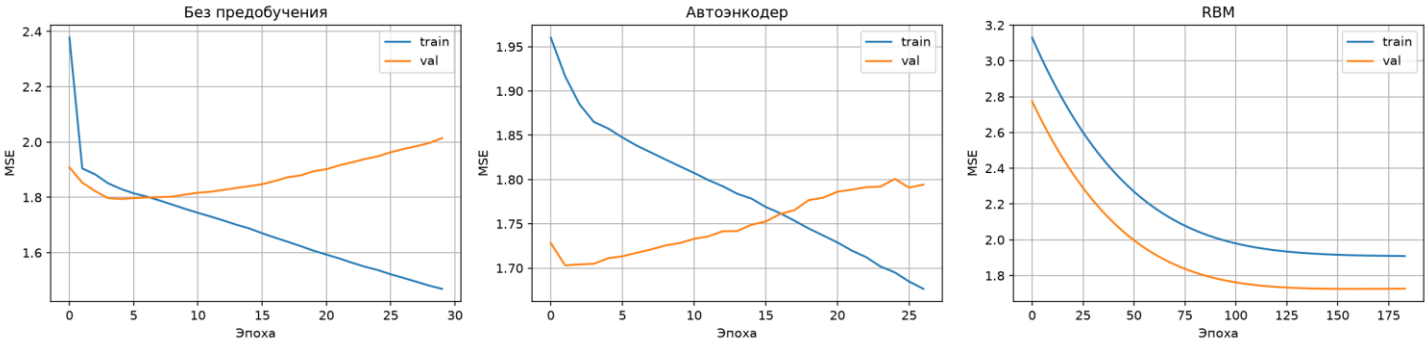
Результат

C:\Users\arosl\PycharmProjects\IAD4\.venv\Scripts\python.exe
C:\Users\arosl\PycharmProjects\IAD4\main.py
Выбор датасета
1 - Forest Fires (регрессия)
2 - Rice (классификация)
Введите номер датасета (1 или 2): 1
Выбран Forest Fires
Датасет: Forest Fires
Обучающая: (413, 29)
Тестовая: (104, 29)
Размерность: 29
Шаг 1. Без предобучения
Без предобучения
MSE: 12084.6923828125
MAE: 19.849445343017578
MAPE: 86.01216673851013
Шаг 2. Автоэнкодерное предобучение
Предобучение слоя 1 dim = 64
Выход автоэнкодера: (413, 64)
Предобучение слоя 2 dim = 32

Выход автоэнкодера: (413, 32)
Предобучение слоя 3 dim = 16
Выход автоэнкодера: (413, 16)
Автоэнкодер
MSE: 12084.2802734375
MAE: 19.876615524291992
MAPE: 90.16035795211792
Шаг 3. RBM-предобучение
RBM-предобучение слоя 1 dim = 64
  выход RBM: (413, 64)
RBM-предобучение слоя 2 dim = 32
  выход RBM: (413, 32)
RBM-предобучение слоя 3 dim = 16
  выход RBM: (413, 16)
RBM
MSE: 12106.1728515625
MAE: 19.778942108154297
MAPE: 78.62880825996399
Сохранено: training_curves_forest.png

Метрика	Без предобучения	Автоэнкодер	RBM
MSE	12084.692383	12084.280273	12106.172852
MAE	19.849445	19.876616	19.778942
MAPE, %	86.012167	90.160358	78.628808

Process finished with exit code 0



По MSE автоэнкодер и базовая модель показали практически идентичный результат , в то время как RBM немного уступила (~0.18%).По MAE RBM показала наименьшую абсолютную ошибку (19.779), немного обогнав базовую модель (19.849). По MAPE RBM показывает наилучший результат (78.6%), существенно опережая базовую модель (86.0%) и автоэнкодер (90.2%). Это означает, что RBM-модель наиболее точно предсказывает значения для типичных (небольших) пожаров, где относительная ошибка наиболее чувствительна. Метрика MSE сильно завязана на выбросы (крупные лесные пожары). Из-за малого размера тестовой выборки (104 объекта) одна крупная ошибка может сильно сместить MSE. Поэтому MAE и MAPE в данном случае являются более информативными метриками, и по ним RBM-

предобучение выигрывает.

КЛАССИФИКАЦИЯ

Выбор датасета
1 - Forest Fires (регрессия)
2 - Rice (классификация)
Введите номер датасета (1 или 2): 2
Выбран Rice
Датасет: Rice
Обучающая: (3048, 7)
Тестовая: (762, 7)
Размерность: 7
Классов: 2 ['Cammeo', 'Osmancik']
Шаг 1. Без предобучения
Без предобучения
Accuracy: 0.9173228346456693
F1 macro: 0.9152560624061092
F1 weighted: 0.9171665241401564

	precision	recall	f1-score	support
Cammeo	0.91	0.89	0.90	326
Osmancik	0.92	0.94	0.93	436
accuracy			0.92	762
macro avg	0.92	0.91	0.92	762
weighted avg	0.92	0.92	0.92	762

[[290 36]
[27 409]]
Шаг 2. Автоэнкодерное предобучение
Предобучение слоя 1 dim = 64
Выход автоэнкодера: (3048, 64)
Предобучение слоя 2 dim = 32
Выход автоэнкодера: (3048, 32)
Предобучение слоя 3 dim = 16
Выход автоэнкодера: (3048, 16)
Автоэнкодер
Accuracy: 0.916010498687664
F1 macro: 0.9139469226425748
F1 weighted: 0.91587059522698

	precision	recall	f1-score	support

```
      Cammeo       0.91      0.89      0.90       326
    Osmancik       0.92      0.94      0.93       436

    accuracy                           0.92       762
   macro avg       0.92      0.91      0.91       762
weighted avg       0.92      0.92      0.92       762

[[290  36]
 [ 28 408]]
Шаг 3. RBM-предобучение
RBM-предобучение слоя 1 dim = 64
  выход RBM: (3048, 64)
RBM-предобучение слоя 2 dim = 32
  выход RBM: (3048, 32)
RBM-предобучение слоя 3 dim = 16
  выход RBM: (3048, 16)
RBM
Accuracy: 0.9146981627296588
F1 macro: 0.9121813181844778
F1 weighted: 0.9143274646958726
              precision    recall  f1-score   support

      Cammeo       0.93      0.87      0.90       326
    Osmancik       0.91      0.95      0.93       436

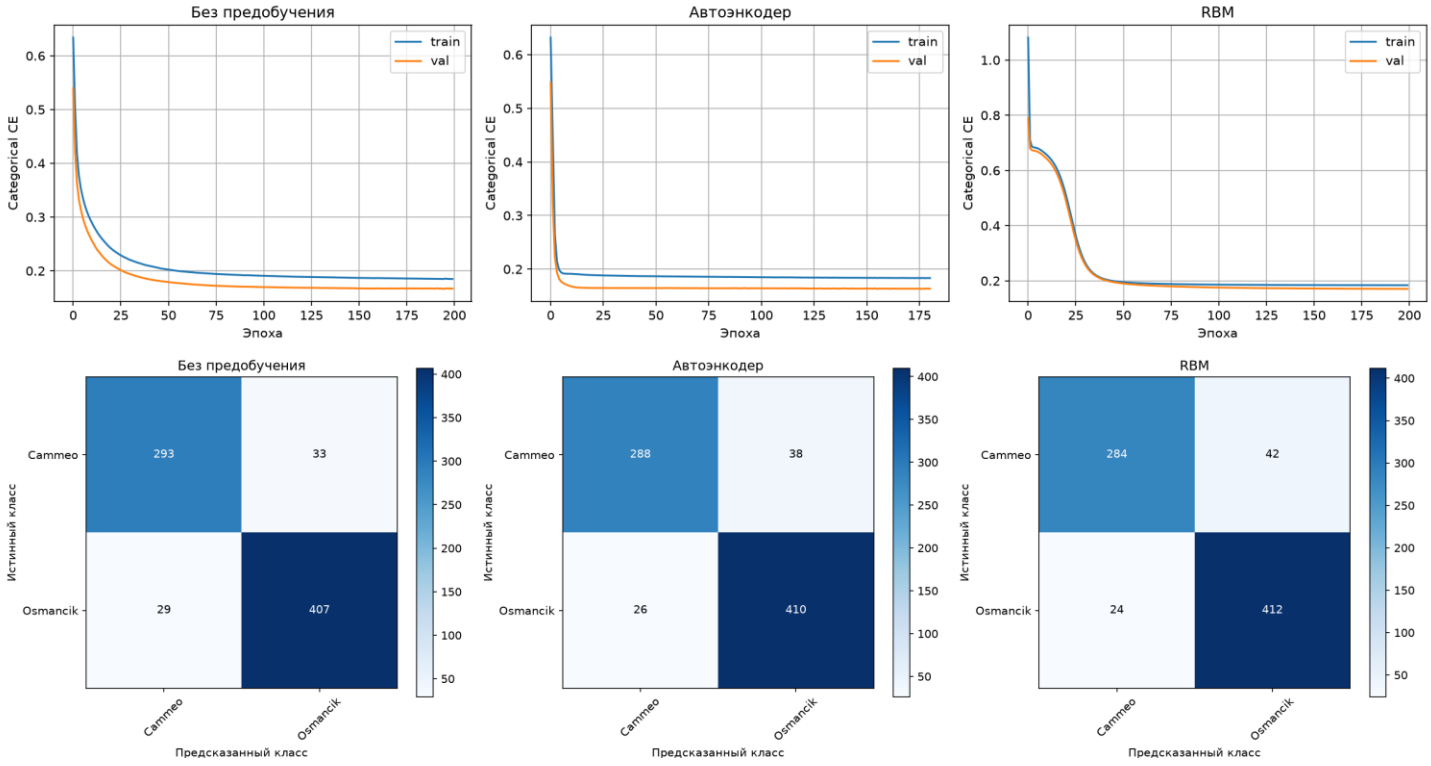
    accuracy                           0.91       762
   macro avg       0.92      0.91      0.91       762
weighted avg       0.92      0.91      0.91       762

[[284  42]
 [ 23 413]]
Сохранено: training_curves_rice.png
Сохранено: confusion_matrices.png
      Метрика  Без предобучения  Автоэнкодер      RBM
     Accuracy          0.917323     0.916010 0.914698
   F1 (macro)          0.915256     0.913947 0.912181
F1 (weighted)          0.917167     0.915871 0.914327

Process finished with exit code 0
```

Все три модели показывают практически одинаково высокое качество .

Базовая модель без предобучения оказалась немного лучше (Accuracy 91.73%). RBM-модель (Accuracy 91.47%) немного уступает, однако обратим внимание на матрицу ошибок: RBM лучше распознает мажоритарный класс Osmancik (Recall 0.95 против 0.94 у базовой), но делает больше ложных срабатываний на классе Cammeo (42 против 36).В данном случае предобучение не дало выигрыша, так как датасет Rice достаточно большой (3048 объектов), простой (7 признаков) и хорошо разделимый. Случайная инициализация Keras уже обеспечивает отличную стартовую точку, а предобучение лишь слегка «сглаживает» пространство весов.



Вывод: в ходе работы было выполнено сравнение трёх подходов к обучению нейронной сети. Эффективность предобучения сильно зависит от типа задачи и объема данных.